Group 38 Progress Report: RealWaste Classification

Junghoon Park, Sihoon Park
{parkj189,parks138}@mcmaster.ca

1 Introduction

The rapid global increase in waste generation significantly contributes to environmental impacts such as pollution, ecosystem degradation, and climate change. Recycling is an important solution to minimize these impacts, yet its effectiveness is prone to contamination in recycling streams. During the recycling process, misclassification often contaminates entire batches, rendering them unusable.

Our project addresses this problem by developing an automated waste classification system using machine learning. By leveraging Convolutional Neural Networks (CNNs), we aim to create a model that can accurately classify images of waste into their corresponding categories. This automated approach can reduce the reliance on traditional human labor, which may cause human error, improve sorting accuracy, lower operational costs, and increase overall efficiency. In addition, automated classification supports more effective recycling practices, helping to conserve resources and reduce environmental impact.

2 Related Work

After reviewing previous attempts on waste classification, it is revealed that training a Convolutional Neural Network (CNN) from scratch risks overfitting and poor generalization due to the limited number of inputs (Arcioni, 2025; Arvidsson, 2025; chamodperera87, 2023). To address this issue, the original paper for the dataset (Single et al., 2023) and related research (Yu, 2024; Arcioni, 2025) implement fine-tuning. They use pre-trained models, such as Inception V3, VGG-16, and ResNet50, along with fine-tuning and data augmentation to prevent overfitting and to achieve higher accuracy. Building on this approach, our task focuses on fine-tuning and evaluating an extended range of pre-trained models, including EfficientNetV2, ResNet50, DenseNet121, ConvNeXtV2, and Inception V3. Furthermore, we incorporate the optimization findings(Ilya and Frank, 2019; Tong et al., 2018). Their research highlights that standard Adam implementations weight decay sub-optimally learning to less generalization and demonstrate that AdamW, especially when paired with a learning rate scheduler, minimizes the issue and improves the performance. By adopting this strategy and systematic approach to hyperparameter tuning, we aim to maximize the performance and exceed both the published baseline where the Inception V3 model achieved a precision of 91.34%, recall of 87.73%, and F1 score of 90.25% and the highest results where Resnet50 model achieved a F1 score of 93.88% by (Arcioni, 2025).

3 Dataset

3.1 Data Description

The dataset consists of 4,752 images in JPG format which are categorized into nine classes with a single label per image: Cardboard, Food Organics, Glass, Metal, Miscellaneous Trash, Paper, Plastic, Textile Trash, and Vegetation.

Images were captured from the Whyte's Gully Waste and Resource Recovery Centre's landfill site located in Wollongong, New South Wales, Australia. Waste items were from municipal waste collection comingle and contaminated to reflect real-word conditions.

Each image was originally scaled to 524×524 image resolutions to better distinguish the object features, including commingled state of material types and transparent objects and reach better classification accuracy.

The class distribution is as follows: Cardboard: 351; Food Organics: 411; Glass: 420; Metal: 790; Miscellaneous Trash: 495; Paper: 500; Plastic: 921; Textile Trash: 318; Vegetation: 436.

This distribution shows a significant class imbalance which is later handled by weighted CrossEntropyLoss: the most represented class (Plastic) appears almost three times more often than the least represented class (Textile Trash).

3.2 Preprocessing

For preprocessing, all images were resized to 224×224 pixels using `torchvision.transforms` to match the input requirements of pre-trained CNN models such as ResNet50 and ConvNeXtV2, and subsequently converted to tensors for model training. Additionally, we applied ImageNet normalization with a mean of [0.485, 0.456, 0.406] and a standard deviation of [0.229, 0.224, 0.225] across the entire dataset. This step is essential for fine tuning because our pretrained models trained on ImageNet expect inputs normalized to this specific distribution.

3.3 Annotation

The dataset was pre-annotated by the provider. Initially, each image was labeled under three broad material types: divertible organics, recyclable inorganic, and general waste. These labels were later refined into the nine specific classes: Cardboard, Food Organics, Glass, Metal, Miscellaneous Trash, Paper, Plastic, Textile Trash, and Vegetation.

As part of our data loading pipeline, we numerically encoded these labels. Our custom Load_Transform_Dataset class uses a class_dict to map each string label to a unique integer.

Index	Class Label
0	`Cardboard`
1	`Food Organics`
2	`Glass`
3	`Metal`
4	`Miscellaneous Trash`
5	`Paper`
6	`Plastic`
7	`Textile Trash`
8	`Vegetation`

Table 1: Numerical Label Encoding for Classes

4 Features

4.1 Input

The direct input to our models is a batch of preprocessed image tensors with the shape (batch_size, 3, 224, 224) where each tensor corresponds to a 224 x 224 RGB image.

4.2 Representation Learning

Instead of training a CNN model from scratch, which would likely lead to overfitting on our relatively small dataset, we leverage transfer learning by pre-trained models on the large scale ImageNet dataset. The frozen convolutional backbone layers act as sophisticated feature extractors which transform raw pixel data of the input into high level feature vectors. Critically, we then fine-tune the final convolutional block which allows the models to adapt to the RealWaste dataset while retaining the general visual features learned from ImageNet.

- ResNet50: Extraction through convolutional and residual layers with skip connections. The final Global Average Pooling (GAP) layer produces a 2048-dimensional feature vector.
- DenseNet121: Extraction through densely connected Convolutional layers. The final GAP layer produces a 1024-dimensional feature vector.
- ConvNeXtV2-base: Extraction through ConvNeXt blocks enhanced with Global Response Normalization (GRN) layers, with weights pre-trained using FCMAE. The final GAP layer produces a 1024-dimensional feature vector
- EfficientNetV2: Extraction through MBConv and Fused-MBConv blocks. The final GAP layer produces a 1280-dimensional feature vector.

4.3 Data Augmentation

To avoid overfitting and increase the data diversity, we apply a series of random augmentation using torchvision.transforms. For each image in the original training set, we generate three augmented copies which are implemented via transform. Compose includes RandomHorizontalFlip(), RandomVerticalFlip(), RandomRotation(), ColorJitter(), and RandomResizeCrop(). We selected augmentation parameters that balance data variety with preservation of meaningful visual features. Specifically, we constrained RandomResizedCrop's parameter to (0.75, 1.0) to prevent excessive cropping which might remove critical portions of waste objects ensuring objects remain

the dominant subject. Similarly, ColorJitter was set with moderate parameters (brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1) to preserve distinctive color features essential for classification while introducing sufficient lighting variance for diversity.

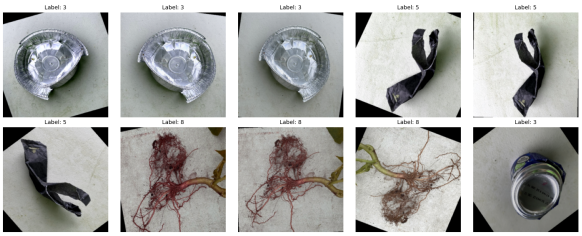

Figure 1: Sample Augmented Data

5 Implementation

5.1 Model Used

We implemented transfer learning with fine-tuning using four pretrained CNN models from torchvision and timm packages. For each model, we load the model with its default pre-trained weights:

Model Name	Loading Function
EfficientNetV2-S	`torchvision.models.efficientnet_v2_s`
ResNet50	`torchvision.models.resnet50`
DenseNet121	`torchvision.models.densenet121`
ConvNeXtV2-Base	`timm.create_model("convnextv2_base")`

Table 2: Pre-trained Models Loading.

5.2 Partial Fine-Tuning Implementation

Our partial fine-tuning strategy is that we initially freeze all parameters of the pre-trained model using param.requires_grad = False for all parameters. Then, we selectively unfreeze only the parameters of the final classification layer and the last convolution block while leaving the other convolutional backbone frozen to both adapt to RealWaste data and keep the general visual features learned from ImageNet:

Model	Replace Classifier	Unfreeze Last Block
EfficientNetV2-S	`model.classifier[-1] = nn.Linear(...)`	`model.features[-1]`
ResNet50	`model.fc = nn.Linear(...)`	`model.layer4`
DenseNet121	`model.classifier = nn.Linear(...)`	`model.features.denseblock4`
ConvNeXt V2	`model.reset_classifier(NUMOFCLASS)`	`model.stages[-1]`

Table 3: Partial Fine-Tuning Implementation Details

5.3 Training Configuration

Loss Function: Our dataset has a significant class imbalance as described earlier. The stratify parameter used during our data split ensures the same imbalance proportion and not make the imbalance worse in different sets. To address this, we use a weighted nn.CrossEntropyLoss. The weights are computed using compute_class_weight from sklearn.utils.class_weight with a balanced setting.

Optimizer: We use AdamW optimizer (optim.AdamW) with weight_decay = 1e-2 for regularization. For every model, we implemented two learning rates:

- Final Convolutional Block: we use a smaller learning_rate = 1e-5 to fine tune pre-trained features carefully.
- Classifier Layer: A higher c_learning_rate = 1e-4 to allow fast adaption to the RealWaste dataset.

Learning_Rate_Scheduler: We use ReduceLROnPLateau scheduler (optim.lr_scheduler.ReduceLROnPLateau) with patience=2 and factor=0.7 which reduces the learning rate when validation loss plateaus to allow fine-grained optimization in later epochs.

All models were trained for epochs 30 and BATCH_SIZE of 64. Additionally, the training and validation loss and accuracy were plotted for each epoch to monitor its performance.

5.4 Ablations

Our choices of a partial fine tuning strategy and optimizer with scheduler was determined through exporting different implementations.

We first experimented with linear probing which trains only the final classifier layer while keeping the entire backbone frozen. This approach yielded poor performance below the baseline indicating the general ImageNet features were insufficiently adapted to the RealWaste dataset.

Next, we experimented with full fine-tuning, unfreezing all layers of the pre-trained models. While this yielded significant performance improvements, it was computationally expensive and not practical, especially for large models such as ConvNeXtV2 due to the substantial GPU memory and training time requirement.

As a result, we chose a partial fine-tuning as our final approach. By unfreezing only the final convolutional block and the new classifier layer, the models effectively adapted their high level representation to the RealWaste dataset while maintaining computational efficiency and general-purpose features learned from ImageNet.

Regarding our optimization strategy, we initially implemented the standard Adam optimizer. Even though Adam optimizer yielded respectable results with the ConvNeXtV2 model outperforming our initial baseline, we observed a noticeable improvement in generalization performance after replacing it with AdamW. For learning rate scheduling, we first experimented by using Cosine Annealing. However, we observed that this strategy reduced the final accuracy and was not suitable for our models with our initial limited training budget of 15 epochs. To address this, we shifted to ReduceLROnPlateau which allows the model to maintain a higher learning rate for longer periods and only decays the learning rate when validation loss plateaus. We provide a more detailed reflection on these deviations from our original plan in the Progress section.

6 Progress

Throughout the progressive development of this project, we shifted our focus from exploring entirely new architectures to optimizing the pretrained CNN models already in use. Since these models already exhibited strong baseline performance, we determined that further refinement would be more impactful than simply testing additional predefined CNN models. Consequently, our progress centered on maximizing performance through a range of techniques as planned in the progress report, including more diverse data augmentation strategies, the adoption of more advanced optimizers paired with learning rate schedulers.

6.1 Optimizer

In our Progress Report, we utilized the standard Adam optimizer which yielded respectable results with the ConvNeXtV2 model outperforming our initial baseline. However, we identified potential room for improvement based on research indicating that standard Adam implements weight decay suboptimally by coupling it with the gradient update which often leads to poorer generalization. To address this, we transitioned to more advanced optimizer AdamW (Adam with Decoupled Weight Decay) with no such issue that generally performs better than the standard Adam in most tasks. This modification showed significant improvements across our pretrained models, most notably boosting the final accuracy of our ConvNeXtV2 model from 90.95% to 92.86% and our EfficientNetV2-S model from 73.68% to 76.89%. This significant performance gain confirms that correcting the weight decay implementation was critical for our dataset and validates our decision to utilize AdamW optimizer for the final evaluation.

In addition to switching optimizers, we experimented with different weight decay values for AdamW, specifically testing with 1e-4,1e-3, and 1e-2. We observed that the lower weight decay values 1e-4 and 1e-3 provided insufficient regularization. Ultimately, we found that 1e-2 yielded the best performance among them as it resulted in a more stable training curve and higher final accuracy. Therefore, we selected the weight decay of 1e-2 for the final evaluation.

6.2 Learning Rate Scheduler

Guided by best practices in transfer learning research, we identified that pairing the AdamW optimizer with a learning rate scheduler is critical for maximizing performances of the models.

Initially, we implemented Cosine Annealing(CosineAnnealingLR), which decreases the learning rate following a cosine curve over the epochs of training. However, this strategy yielded poorer results for our specific constraints. With our training budget limited to 15 epochs, we observed that Cosine Annealing reduced the learning rate too continuously. This prevented the model from fully leveraging higher learning rates during the early and mid phases of training. As a result, the model's learning slowed even though it was still capable of learning rapidly.

To address this, we adopted the ReduceLROnPlateau scheduler. Unlike Cosine Annealing, this scheduler maintains the high learning rate and only decays it when the validation loss actively stops improving. We calibrated the parameters to patience = 2 and factor=0.7 specifically for our limited training budget, we ensured the model maximized its learning potential before decaying. This adjustment resulted in improving the overall accuracy of the models especially for our ConvNeXtV2 model from 92.86% to 93.52%. Therefore, we decided to use ReduceLROnPlateau as the scheduler for the final evaluation.

6.3 Augmentation Techniques

To further increase the generalization, we recognized that our initial augmentation that are limited to basic augmentation techniques, horizontal flips, vertical flips, and rotations, might be insufficient

for generating diverse training samples. To address this, we introduced RandomResizedCrop and ColorJitter transformations. Augmentation parameters were carefully chosen to balance data diversity with preservation of meaningful visual features. Specifically, RandomResizedCrop was constrained to a scale of (0.75, 1.0) to avoid excessive cropping that could remove critical portions of waste objects, ensuring that objects remain the dominant subject. ColorJitter was applied with moderate parameters (brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1), introducing sufficient lighting and color variation for diversity while maintaining distinctive color cues essential for classification. These modifications yielded a slight improvement in overall performance, with our best model ConvNeXtV2 achieving a final accuracy increase from 93.52% to 94.3%.

6.4 Epochs

Finally, for our ablation experiments involving optimizers and augmentations, we intentionally limited the training budget to 15 epochs due to time constraints. However, the validation loss curves indicated that performance was still improving with some fluctuations suggesting that the models possessed further capacity to learn. To fully exploit this potential, we extended the training epochs from 15 to 30 for our final evaluation. This extension led to noticeable improvements across all models. The specific final accuracy metrics resulting from the extended training are reported in the Results and Evaluation section.

6.5 Challenges Encountered

However, applying strategies did not always lead to a positive outcome. To address the highly inconsistent backgrounds in the dataset, ranging from gray and yellow surfaces to cracks, dirt, and other irregular textures, we tested an AI-based preprocessing approach that replaced all backgrounds with a uniform black color. The goal was to reduce background-induced noise and help the model focus more directly on the objects. After applying this transformation, each training run took less time, likely due to the simplified visual patterns. However, the overall accuracy consistently declined across experiments.

For example, the validation accuracy of EfficientNetV2-S decreased from 76.89% to 69.54% after the background replacement. ResNet50 showed a smaller decline, dropping from 89.71% to 88.24%, while DenseNet121 exhibited a more noticeable reduction from 89.71% to 82.77%. Lastly, ConvNeXtV2 fell from 92.86% to 87.82%. Across all four architectures, the validation accuracy consistently declined despite the shorter training time per epoch, suggesting that the uniform black backgrounds removed some features that were important for generalization.

Overall, this experiment suggests that completely replacing image backgrounds can simplify the training process but may reduce model accuracy.

Additionally, as proposed in our Progress Report, we explored a deeper fine tuning strategy by unfreezing the last two convolutional blocks rather than unfreezing the final blocks with the assumption that this would allow the models to learn more abstract features of our dataset. However, this approach resulted in a significant degradation in performance across all models, likely because our parameters and techniques were not optimized which may have caused catastrophic forgetting. Consequently, we reverted to our initial approach of unfreezing only the final convolutional block.

7 Results and Evaluation

7.1 Data Splits

Our models were evaluated using a train, validation, and test set split approach (Training set 80%, Validation Set 10% and Test Set 10%). To avoid the class imbalance problem, stratified sampling was used during the split. The use of stratified sampling ensures the relative distribution of each class is approximately the same across all splits, preventing some classes from being under or over represented. This also helps to make the evaluation fair across all classes and avoids misleading results caused by class imbalance.

7.2 Evaluation Strategy

We first calculated overall accuracy to measure the general performance of each model. In addition, confusion matrices were used to visualize how well each model predicted each class. The use of confusion matrices allows us to capture whether there was any specific class where the models performed poorly or tended to be confused with other classes. Normalizing the confusion matrices to percentages made it easier to interpret performance across classes.

To provide more detailed information, we also

printed the precision, recall, and F1-score for each class. Each trained model was evaluated on the test set to collect the above predictions, accuracy, confusion matrices, and class-specific performance measures. This evaluation on unseen test data ensures that the reported metrics reflect the generalization ability of each model.

Finally, to provide a clear visual comparison, we added a bar plot showing the accuracy of each model. This allows readers to quickly and easily understand the relative performance of all models, see which model achieved the highest accuracy, and also observe differences in performance across more challenging or minority classes.

7.3 Result

To evaluate model performance, we used the test dataset which consists of 475 samples. All models were trained under identical conditions using the same data splits, loss function, and optimizer settings to ensure a fair comparison. The evaluation metrics include Accuracy, Precision, Recall, and F1-Score.

According to the final test set results summarized in the Table 4, among the tested models, ConvNeXtV2 achieved the highest performance with an accuracy of 95.58%, precision of 95.62%, recall of 95.58%, and F1 score of 95.57%. This was followed by ResNet50 and DenseNet121 which also demonstrated strong result with F1 score of 92.86% and F1 score of 90.49%, respectively. However, EfficientNetV2_S showed significantly lower generalization performance with an F1 score of 76.69%. Compared to our progress report, these results of our final evaluation with modification indicate that our models performed significantly better overall.

The baseline model reported in the original paper used Inception V3, which achieved a precision of 91.34%, recall of 87.73%, and F1 score of 90.25%. Compared to this baseline, our models, ConvNeXtV2, ResNet50, and DenseNet121, achieved higher F1 scores, recalls, and precisions. Especially, our best model, ConvNeXtV2, achieved a F1 score of 95.58% which is 5.33% higher than the baseline.

Additionally, our final evaluation results showed that ConvNeXtV2 model with a F1 score of 95.58% not only surpassed the baseline done by Inception V3, but also outperformed the highest F1 score of 93.88% by Resnet50 from the previous studies on our dataset.

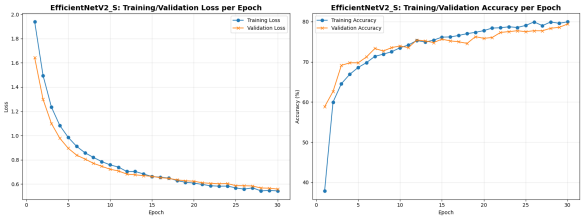

Figure 2: EfficientNetV2: Training/Validating Loss And Accuracy

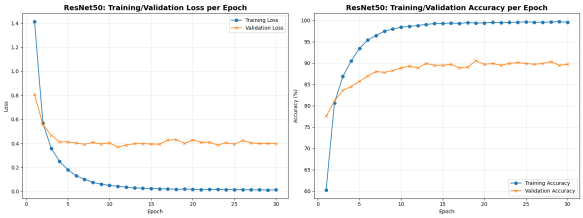

Figure 3: ResNet50: Training/Validating Loss And Accuracy

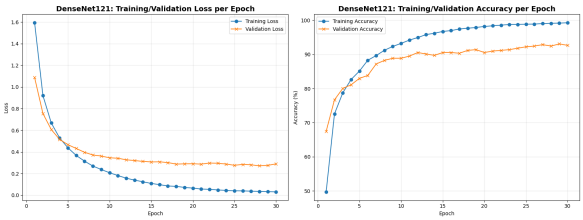

Figure 4: DenseNet121: Training/Validating Loss And Accuracy

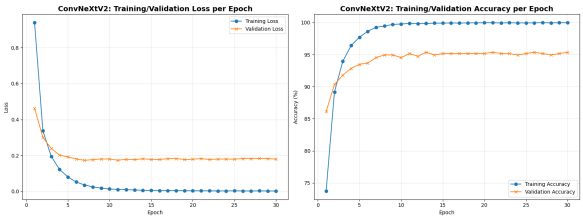

Figure 5: ConveNeXtV2: Training/Validating Loss And Accuracy

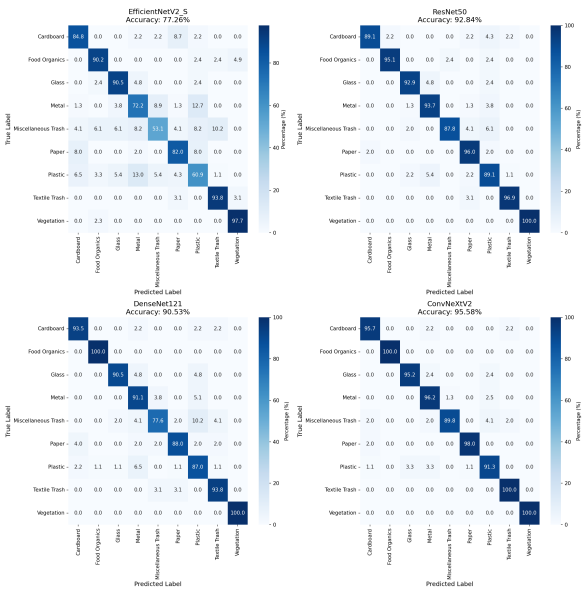

Figure 6: Confusion Matrix

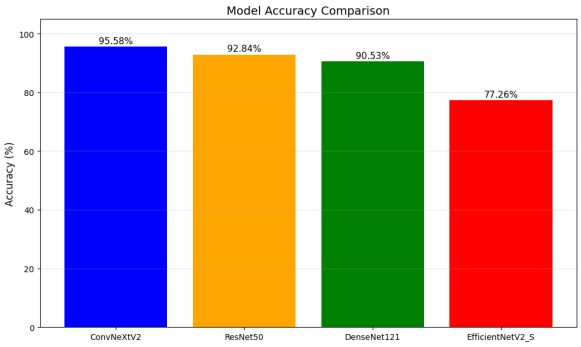

Figure 7: Accuracy Comparison Plot

Figure 8: Miscellaneuous Class: Ambiguous Example

Model	Acc.	Prec.	Rec.	F1	Samples
ConvNeXtV2	95.58%	95.62%	95.58%	95.57%	475
ResNet50	92.84%	93.02%	92.84%	92.86%	475
DenseNet121	90.53%	90.61%	90.53%	90.49%	475
EfficientNetV2-S	77.26%	76.76%	77.26%	76.69%	475

Table 4: Model Evaluation Results on the Test Dataset.

8 Error Analysis

Across all models, the lowest accuracy was consistently observed for the Miscellaneous Trash label. Since this pattern appears in every model, it suggests that the issue comes from the dataset rather than a specific model's weakness. The category itself is ambiguous, and upon inspecting the data, we found many instances of single-use plastic items and other mixed-material waste. For example, a plastic spoon shown in Figure 7 is annotated as Miscellaneous Trash. The heterogeneous nature of these images makes classification difficult, so improving the model alone is unlikely to significantly increase accuracy. Therefore, unless the dataset is refined or more clearly labeled, these errors are hard to resolve.

Following the Miscellaneous Trash, Plastic was the second-lowest performing label across all models. There was a noticeable pattern observed from the confusion matrix, that the models are frequently confused with plastic and metal.

Notably, in all models except EfficientNetV2_S, a consistent pattern was observed. Among misclassified Metal samples, approximately half were predicted as Plastic, and conversely, among misclassified Plastic samples, roughly half were predicted as Metal. This indicates that Metal and Plastic are the primary sources of confusion for each other across these models.

In terms of quantitative performance, EfficientNetV2_S misclassified 27.8% of Metal samples, with 12.7% being predicted as Plastic. Similarly, 39.1% of Plastic samples were incorrect, with 13% misclassified as Metal. ResNet50 significantly reduced these error rates: Metal misclassification dropped to 6.3% (with 3.8% as Plastic), and Plastic misclassification fell to 10.9% (with 5.4% as Metal). DenseNet121 showed comparable results, with 8.9% of Metal samples misclassified (5.1% as Plastic) and 13% of Plastic samples misclassified (6.5% as Metal). ConvNeXtV2 achieved the lowest error rates, with only 3.8% of Metal samples misclassified (2.5% as Plastic) and 8.7% of Plastic samples misclassified (3.3% as Metal).

This confusion seems to be caused by similar visual features, such as reflective or shiny surfaces. To address this issue, a more diverse set of images is required, for example, by including photos taken under darker or varied lighting conditions to reduce reflections.

References

Lorenzo Arcioni. 2025. Waste Image Classification. https://www.kaggle.com/code/lorenzoarcioni/waste-image-classification. Kaggle notebook.

Joakim Arvidsson. 2025. Simple CNN Classifier of Waste Products. https://www.kaggle.com/code/joebeachcapital/simple-cnn-classifier-of-waste-products. Kaggle notebook.

chamodperera87. 2023. Cnn for image classification real waste. https://www.kaggle.com/code/chamodperera87/cnn-for-image-classification-real-waste#Training.

Loshchilov Ilya and Hutter Frank. 2019. Decoupled weight decay regularization. *https://arxiv.org/pdf/1711.05101*.

Sam Single, Saeid Iranmanesh, and Raad Raad. 2023. Realwaste: A novel real-life data set for landfill waste classification using deep learning. *Information*.

He Tong, Zhang Zhi, Zhang Hang, Zhang Zhongyue, Xie Junyuan, and Li Mu. 2018. Bag of tricks for image classification with convolutional neural networks. *https://arxiv.org/pdf/1812.01187*.

Peiran Yu. 2024. Garbage sorting and processing based on convolutional neural network. *ResearchGate Preprint*.